

Model Wars: RLBot ML Competition

Participant Technical Specification & Submission Guide

Model Wars Tournament Committee

October 10, 2026

Abstract

This document outlines the complete technical requirements, model architecture contract, development environment setup, and submission verification pipeline for the Model Wars Rocket League Machine Learning Tournament. Participants develop an autonomous machine learning policy to compete in 1v1 Soccar matches powered by RLBot v5. All submissions are automatically evaluated and cryptographically validated via continuous integration.

Contents

1	Competition Overview & Architecture	1
1.1	Standard Observation Contract (13 Dimensions)	2
1.2	Neural Policy Architecture (26,181 Parameters)	2
1.3	Action Decoding Scheme	2
2	Environment Setup & Local Testing	3
2.1	Prerequisites	3
2.2	Local Environment Initialization	3
2.3	Executing Local Verification	3
3	Submission Rules & Integrity Policies	3
3.1	Submission Criteria Checklist	3
4	Step-by-Step GitHub Submission Guide	4
4.1	Step 1: Create Your Private Submission Repository	4
4.2	Step 2: Add Tournament Organizers as Collaborators	4
4.3	Step 3: Clone & Initialize Your Bot Identity	4
4.4	Step 4: Train and Seal Checkpoint	4
4.5	Step 5: Document Results and Push	4
5	Continuous Integration & Verification Feedback	5
5.1	Interpreting CI Badges	5
5.2	Common Validation Errors and Remedies	5

1 Competition Overview & Architecture

The Model Wars tournament evaluates participant-trained neural network controllers in head-to-head 1v1 Rocket League matches. Bots interact with Rocket League via the official **RLBot v5** framework, receiving real-time physics state observations and outputting native controller inputs at variable tick frequencies.

1.1 Standard Observation Contract (13 Dimensions)

Models receive exactly **13 finite float32 values** per decision callback, strictly structured as detailed in Table 1. All car-relative displacements are projected onto the car’s local axes (pitch, yaw, and roll) and normalized by standard game constants.

Index	Feature Name	Normalization	Description
0	Ball Forward	$x/6000$ UU	Car-relative ball displacement along longitudinal axis.
1	Ball Right	$y/6000$ UU	Car-relative ball displacement along lateral axis.
2	Ball Up	$z/6000$ UU	Car-relative ball displacement along vertical axis.
3	Prediction Forward	$x_{\text{pred}}/6000$ UU	Projected 2-second RLBot trajectory slice (forward).
4	Prediction Right	$y_{\text{pred}}/6000$ UU	Projected 2-second RLBot trajectory slice (lateral).
5	Prediction Up	$z_{\text{pred}}/6000$ UU	Projected 2-second RLBot trajectory slice (vertical).
6	Ball Distance	$\ \mathbf{d}\ /6000$ UU	Euclidean car-to-ball distance in 3D space.
7	Car Speed	$\ \mathbf{v}\ /2300$ UU/s	Magnitude of car linear velocity vector.
8	Ball Present	Binary $\{0, 1\}$	Sensor mask indicating ball presence.
9	Prediction Valid	Binary $\{0, 1\}$	Sensor mask indicating valid trajectory packet slice.
10	Horizon Offset	$\Delta t/2.0$	Normalized prediction time offset: $(t_{\text{slice}} - t_{\text{now}})/2$.
11	Slice Offset	$\Delta t_{\text{first}}/(1/120)$	Offset relative to first trajectory packet slice.
12	Callback Δt	$\Delta t/(1/60)$	Elapsed time between consecutive callbacks (0 on first tick).

Table 1: The 13-Dimensional Input Observation Vector.

1.2 Neural Policy Architecture (26,181 Parameters)

To ensure fair computational bounds, the standard tournament baseline requires the exact **13D causal GRU architecture**:

$$\mathbf{x} \in \mathbb{R}^{13} \xrightarrow{\text{Linear}(13 \rightarrow 64)} \mathbf{z}_1 \xrightarrow{\text{ReLU}} \mathbf{z}_2 \xrightarrow{\text{GRU}(64 \rightarrow 64)} \mathbf{h} \xrightarrow{\text{Linear}(64 \rightarrow 5)} \mathbf{y}$$

where $\mathbf{y} = [\mathbf{m}_{1:4}, s]$ denotes:

- Mode logits $\mathbf{m} \in \mathbb{R}^4$: Selects from discrete controller action primitives.
- Continuous steering $s \in [-1, 1]$: Transformed via $\tanh(\cdot)$ activation.
- Total trainable parameters: **26,181**.

1.3 Action Decoding Scheme

Output mode probabilities determine controller signals according to Table 2.

Mode	Behavior	Throttle	Steering	Pitch	Jump
0	Neutral	0.0	0.0	0.0	False
1	Chase	1.0	Continuous $s \in [-1, 1]$	0.0	False
2	Single Jump	0.0		0.0	True
3	Front Dodge (Flip)	0.0		-1.0	True

Table 2: Native Action Mode Table. Yaw, roll, boost, and handbrake default to 0/False in baseline.

2 Environment Setup & Local Testing

2.1 Prerequisites

Ensure the following tools are installed on your workstation:

1. **Python 3.12 (64-bit)**: Enabled with system PATH access.
2. **Rocket League**: Installed via Epic Games Store or Steam.
3. **Official RLBot v5 Launcher**: Downloaded from <https://rlbot.org/v5/>.

2.2 Local Environment Initialization

Run the setup script inside your repository to install pinned dependencies and create the '.venv':

```
# Windows (PowerShell)
.\setup.ps1

# Or double-click SETUP.cmd
```

2.3 Executing Local Verification

Verify your environment and model tensors prior to game execution:

```
# Run numerical and tensor integrity checks
.\.venv\Scripts\python.exe -I -u -B src/preflight.py

# Or double-click CHECK.cmd
```

Launch RLBot v5 using its desktop launcher, click **Add File**, select 'bot.toml', and start a 1v1 match against an opponent bot to confirm live gameplay.

3 Submission Rules & Integrity Policies

Strict Anti-Plagiarism & Baseline Rules

Submissions that use the unmodified starter model weights (V13.pt) or the default starter identifier (shryssssss/v13) are **automatically disqualified and rejected** by the CI validation system.

3.1 Submission Criteria Checklist

Every valid submission must satisfy the following constraints:

- **Trained Weights**: Checkpoint SHA-256 must **not** match 579fa26010140514d2e9b7f0e2871bee569c594b
- **Custom Identity**: 'bot.toml' and 'submission.json' must declare a distinct display name and an 'author/bot' identifier.
- **Training Provenance**: 'submission.json' must specify non-empty experiment metadata: dataset version, training objective, seed, split methodology, and validation scores.
- **Technical Report**: docs/PARTICIPANT_RESULTS.md must document your experiment process, training curves/metrics, and live GUI test results (≥ 50 characters).
- **Security Sandbox**: Code inside src/ must not import restricted libraries (socket, subprocess, requests, urllib) or execute dynamic evaluations (eval, exec).

4 Step-by-Step GitHub Submission Guide

The tournament utilizes private GitHub repositories coupled with GitHub Actions for automated verification.

Workflow Overview

1. Instantiate private repo → 2. Invite Organizer → 3. Initialize bot identity → 4. Train & Seal → 5. Push & CI Verification.

4.1 Step 1: Create Your Private Submission Repository

1. Navigate to the template repository: <https://github.com/Model-Wars/Model-Wars-Base-Repository>.
2. Click the green "Use this template" button → "Create a new repository".
3. Repository Name: <team-name>-rlbot (e.g., apex-rlbot).
4. Set visibility to **Private**. *Do not select Public, or opponents can view your model and code!*
5. Click "Create repository".

4.2 Step 2: Add Tournament Organizers as Collaborators

1. In your private repo, click **Settings** → **Collaborators**.
2. Click "Add people".
3. Search for and invite @Model-Wars (or the designated tournament judge account) with **Read** or **Admin** access.

4.3 Step 3: Clone & Initialize Your Bot Identity

Clone your repository and initialize your custom bot metadata:

```
git clone https://github.com/<your-username>/<team-name>-rlbot.git
cd <team-name>-rlbot

# Initialize custom bot configuration and agent ID in-place:
python tools/kit.py init --name "TeamApex" --agent-id "teamapex/striker"
```

4.4 Step 4: Train and Seal Checkpoint

After conducting your training experiments (via imitation learning or reinforcement learning), seal the resulting PyTorch weights into your repository:

```
# Format: python tools/kit.py seal --checkpoint <path_to_model.pt> --
# provenance <path_to_provenance.json>
python tools/kit.py seal --checkpoint model/V13.pt --provenance docs/
training_provenance.json
```

The 'seal' utility validates tensor finite bounds, updates the exact SHA-256 runtime pin in `src/runtime.py`, and records your training provenance inside `submission.json`.

4.5 Step 5: Document Results and Push

Update docs/PARTICIPANT_RESULTS.md with your measured validation results and commit your work:

```
git add .
git commit -m "Submit trained TeamApex policy"
git push origin main
```

5 Continuous Integration & Verification Feedback

Upon pushing to main, GitHub Actions automatically triggers .github/workflows/validate.yml to run tools/validate_submission.py.

5.1 Interpreting CI Badges

- **[✓ PASS] (Green Checkmark)**: All requirements are fulfilled. Checkpoint is trained, PIN hashes match, tensor architecture is verified, and documentation is present. Your submission is accepted.
- **[× FAIL] (Red Cross)**: One or more checks failed. Click the **Actions** tab on GitHub, inspect the build log, review the failure message (e.g., missing provenance, unchanged starter hash, or invalid shape), fix the issue locally, seal again, and push an update.

5.2 Common Validation Errors and Remedies

Error Message	Resolution
Checkpoint matches unmodified starter weights!	You must replace model/V13.pt with your own trained model weights using <code>kit.py seal</code> .
PIN in src/runtime.py does not match actual SHA256	Run <code>python tools/kit.py seal</code> to update the cryptographic pin in <code>src/runtime.py</code> .
submission.json missing!	Run <code>python tools/kit.py init --name ... --agent-id ...</code> to create your fork metadata.
PARTICIPANT_RESULTS.md missing or too brief	Add detailed experimental notes and validation metrics to <code>docs/PARTICIPANT_RESULTS.md</code> .

Table 3: Troubleshooting common validation failures.